

Fiche individuelle SAE3.02

Groupe Nathan MOUILLE

Nathan Mouille

Table des matières

1. Description du projet.	2
2. Éléments implémentés :	3
• Un serveur Master :	3
• Des clients :	4
• Des routeurs :	4
3. Éléments non implémentés.	5
4. Structure du code	5
• Partie Master :	5
- Partie Configuration / base de données	5
- Partie Interface :	5
- Partie Réseau :	5
- Partie Nettoyage Base de Données :	5
• Partie Routeur :	6
- Partie RSA :	6
- Partie cryptographie :	6
- Partie logique routeur :	6
- Partie Interface :	6
• Partie Client :	6
- Partie Global :	6
- Partie Cryptographie :	6
- Partie Interface :	7
- Partie Threads Réseau :	7
5. Récapitulatif des fonctions	7
- Master :	7
- Routeur :	7
- Client :	8
6. Description des protocoles réseaux:	8
1. Enregistrement auprès du master	8
2. Diffusion des listes aux clients	8
3. Routage en oignon	8
7. Description du cryptage :	9
Principe de l'algorithme :	9
Forces de cet algorithme :	10
Faiblesses et limites de sécurité :	11
Conclusion sur le chiffrement :	11
8. Gestion du projet :	12

1. Description du projet.

Le projet s'intitule « Conception d'une architecture distribuée avec routage en oignon ». Il consiste à mettre en place un réseau composé de plusieurs clients, d'un serveur maître centralisant la gestion et de routeurs intermédiaires chargés de transmettre les messages d'un client à un autre de manière sécurisée et anonyme. Les messages sont relayés via une chaîne de routeurs, ce qui masque l'origine et la destination réelle de la communication aux yeux des nœuds intermédiaires.

L'objectif est de concevoir et d'implémenter un système de communication anonyme entre plusieurs clients à l'aide de routeurs virtuels jouant le rôle de sauts successifs dans le réseau. Le routage en oignon permet de limiter la visibilité des informations sensibles à chaque étape du chemin : chaque routeur ne connaît que l'expéditeur précédent et le destinataire suivant, ce qui réduit les risques de corrélation et d'interception. Le système repose sur une architecture distribuée, avec une gestion des routeurs et des clients centralisée par le serveur maître, et une base de données dédiée pour stocker les informations nécessaires au fonctionnement du réseau (routeurs disponibles et clients enregistrés).

Ce projet cible à la fois les entreprises souhaitant chiffrer et anonymiser leurs échanges internes, et les utilisateurs recherchant davantage de sécurité et de confidentialité dans leurs communications. Le système s'inspire du fonctionnement du réseau Tor, qui repose sur des principes similaires de routage en oignon et de séparation des rôles entre les nœuds du réseau.

2. Éléments implémentés :

Les éléments que j'ai implémentés pour ce projet sont :

- **Un serveur Master :**

Sur le serveur Master, plusieurs fonctionnalités sont implémentées, telles que :

- Un serveur TCP écoutant sur 0.0.0.0:port (le port est configurable dans l'interface)
- Gestion des connexions entrantes avec l'enregistrement des routeurs dans la BDD (le port ainsi que la clé publique) et l'enregistrement des clients dans la BDD (l'adresse IP ainsi que le port utilisé pour communiquer)
- La diffusion périodique aux clients de la liste des routeurs (Routeur : IP : port : clé publique) ainsi que la liste des clients (Client : IP : port)
- Une interface Qt qui affiche en temps réel la liste des routeurs enregistrés (IP, port, clé publique) ainsi que la liste des clients connectés (IP, port). L'interface permet aussi de voir l'IP du serveur et de changer son port d'écoute.
- Intégration de MariaDB qui permet de faire des insertions et des mises à jour des routeurs et clients dans les tables routeurs et clients, elle alimente aussi la colonne date_* et dernier_heartbeat dans les tables routeurs et clients pour une meilleure gestion de la base de données
- Un thread de nettoyage dans le code (cleanup_inactifs) qui supprime de la base de données les routeurs et clients qui sont inactifs au-delà d'un certain délai (HEARTBEAT_TIMEOUT_SEC)

● Des clients :

Sur les clients, plusieurs fonctionnalités sont implémentées, telles que :

- Communication avec le master en TCP, avec l'envoi d'un identifiant (IP : port d'écoute) et un thread de réception qui met à jour les listes de routeurs et de clients dans l'interface Qt.
- Le routage en oignon est implémenté sur le projet avec le choix du nombre de routeurs avant l'envoi du message. Mode aléatoire de choix des routeurs, ou sélection manuelle des routeurs dans la liste distribuée par le master. La construction de l'oignon se fait aussi dans le client avec l'empilement des chiffrements simples basés sur la clé publique de chaque routeur.
- Une Interface Qt avec plusieurs champs à renseigner-t-elle que le champ du master avec l'IP et le port à insérer et le port d'écoute du client sur la partie haute de l'interface. Il y a ensuite la liste des routeurs connus ainsi que la liste des clients connus par le serveur. Il y a une zone de log avec les différents logs qui apparaissent ainsi que les messages. La dernière partie est la zone de saisie des différentes données pour l'envoi d'un message-t-elle que la façon dont la sélection du routage est faite ensuite il y a la zone de saisie de l'IP et le port de l'autre client avec lequel nous voulons communiquer et la dernière zone de saisie est pour le message.
- Il y a aussi une bonne robustesse dans le code avec par exemple un timeout configurable sur la connexion au routeur d'entrée, si le premier routeur ne répond pas il y a ensuite plusieurs essais avec de nouveau chemin si le premier routeur ne répond pas, il y a aussi une gestion des erreurs de connexion avec une fermeture et recréation du socket au routeur d'entrée en cas d'erreurs).
- Pour la réception des messages il y a plusieurs messages-t-elle que sur le listener local sur le port choisi qui reçoit les messages finaux du routeur de sortie et les affiche (Final IP : port ...), il y a aussi un thread de réception sur le socket vers le routeur d'entrée qui affiche les retours.
- Il y a aussi une partie qui ne fonctionne pas mais qui est tout de même incluse où les clients pourraient aussi agir comme routeur, mais pour des raisons de stabilité (crash heap de l'interpréteur sous Windows avec PyQt et threads), cette fonctionnalité n'a pas été gardée dans la version finale. Elle reste disponible sur le GitHub, même si elle n'arrive pas à agir correctement comme routeur.

● Des routeurs :

Sur les routeurs il y a plusieurs fonctionnalités sont implémentées, telles que :

- Il y a l'interface Qt avec plusieurs champs, ceux qui sont en haut permettent de remplir le port ainsi que l'IP du Master, on peut ensuite saisir le port d'écoute du routeur, après avoir inscrit tout ça on peut démarrer la connexion au master, la dernière fenêtre est celle qui affiche les logs réseau du routeur.
- Génération de clés RSA pour chaque routeur généré au démarrage et envoi de la clé publique et du port d'écoute au Master.
- Traitement des messages par oignon avec différentes fonctions, telles que la lecture de la taille, lecture du message chiffré et déchiffrement avec la fonction `simple_dechiffrement`,

puis gestion de si le message est un hop intermédiaire ou un message final. Si le message est un hop intermédiaire le routeur répond OK au précédent hop.

- Il y a aussi un logging vers l'interface de façon thread-safe via `pyqtSignal(str)` pour éviter les accès Qt depuis les threads réseau (qui provoquaient des crashes avec `QTextCursor`).

3. Éléments non implémentés.

Il y a quelques éléments qui n'ont pas pu être implémentés, tels que, comme dit précédemment la mise en place d'un client capable de faire les fonctions d'un routeur dû aux problèmes rencontrés lors de l'exécution du code. Une autre fonction est la discussion entre client avec des identifiants et non avec l'adresse IP et le port, cela permettra une meilleure sécurité car au lieu de devoir savoir l'adresse IP et le port le client devrait juste connaître le pseudo du client avec qui il veut discuter.

4. Structure du code

Voici la structure de mon code Master, routeur et client

- **Partie Master :**

- **Partie Configuration / base de données**

Constantes `DB_CONFIG` qui gère la connexion à la base de données avec `db_conn` et `db_cur`

Fonction `init_db()` qui ouvre MariaDB

- **Partie Interface :**

Classe `MasterWindow (QWidget)` avec deux `QListWidget` pour les routeurs et les clients, méthode `update_view` pour garder les listes actualisés (`routeurs_snapshot`, `clients_snapshot`)

Classe `GuiUpdater (QObject)` avec `update_signal (pyqtSignal)` pour mettre à jour l'interface depuis les threads réseau.

- **Partie Réseau :**

Une partie utilitaire qui se nomme `recv_all (conn, length)` qui permet de lire exactement N octets.

Une fonction `broadcast_routeurs_et_clients()` qui permet de construire les liste des routeurs et des clients, qui envoie ensuite à tous les clients connectés la liste des routeurs et clients et déclenche la mise à jour de l'interface.

Une fonction `handle_routeur(conn,addr)` qui traite un enregistrement de routeur sur le Master et permet de l'enregistrer sur la base de données.

Une fonction `handle_client(conn,addr)` qui enregistre un client sur le Master et permet son enregistrement sur la base de données, cette fonction garde aussi la connexion ouverte pour communiquer les listes des routeurs et clients.

Une fonction `server_thread()` cette fonction est une boucle `accept()`, elle lit un octet d'identification (`b'R'` ou `b'C'`) et lance le bon handler dans un thread correspondant.

- **Partie Nettoyage Base de Données :**

Cette partie fonctionne avec une fonction `cleanup_inactifs()` qui est un thread périodique qui supprime de la base de données les lignes trop anciennes via `dernier_heartbeat` qui permet de voir quand est-ce que le routeur ou client était actif et agir en conséquence.

- **Partie Routeur :**

- **Partie RSA :**

Il y a plusieurs fonctions pour le RSA telle que `est_premier`, `generer_premier`, `pgcd`, `bezout`, `inverse_modulaire`, `generer_cle_rsa`. Ses fonctions seront expliquées plus tard dans la partie cryptage et décryptage.

Il y a aussi une fonction qui génère la clé publique et clé privée avec leur fonction distincte `cle_publice` et `cle_privée`.

- **Partie cryptographie :**

Il y a plusieurs fonctions pour le chiffrement et le déchiffrement qui ne sont pas très compliquées. Il y a `simple_chiffrement (message, n)` et `simple_dechiffrement (message,n)` ce sont des chiffrement symétrique basé sur la valeur `n`. Il y a aussi la fonction `recv_all (conn, length)` pour une lecture fiable du message reçu. il y a aussi une partie sur la communication avec le master avec la fonction `envoyer_cle_publice_au_master (ip_master, port_master, cle_pub, listen_port)` qui après que les bonnes informations ont été renseignées sur l'interface permet l'envoi de la clé publique au master pour un affichage sur son interface.

- **Partie logique routeur :**

Pour la partie logique routeur il y a la fonction `handle_client (conn,addr,log_fn)` qui permet de tout gérer. La fonction lit la longueur du message et le message, il déchiffre le message et le sépare (`first_field | rest,`), il traite ensuite le message de la façon voulu : si `first_field` contient `ip:port` il comprend que c'est un hop intermédiaire et le connecte au routeur suivant, autrement il comprend que cela est un routeur de sortie et envoie `message_final` à `ip_dest:port_dest` ce qui envoie le message au client voulu. Il y a aussi la fonction `serveur_routeur(listen_ip,listen_port,log_fn)` qui crée un thread `handle_client` par connexion au routeur.

- **Partie Interface :**

Pour la partie interface il y a plusieurs fonction utilisés qui permette une bonne interface pour l'utilisateur : il y a la classe `RouteurWindow` : qui a un des informations que l'utilisateurs doit remplir telle que l'IP master, port master, port d'écoute du routeur, Un bouton Connexion master et démarrage routeur ainsi qu'une zone de log. L'interface utilise aussi un signal `log_signal(str)` pour logger depuis les threads et réduire la charge sur le routeur. L'interface utilise aussi une méthode `on_connect_master()` qui lit la configuration que l'utilisateur à inscrit puis envoie la clé publique au master et lance `serveur_routeur` dans un thread.

- **Partie Client :**

- **Partie Global :**

Il y a plusieurs fonctions telles que les listes routeurs, `autres_clients` qui sont partagées et les sockets globaux `sock_master` et `sock_routeur_entree` qui permette de communiquer avec le serveur master et les routeurs.

- **Partie Cryptographie :**

Les fonctions utilisées pour la partie cryptographie sont en partie pareil que pour les routeurs avec `simple_chiffrement`, `simple_dechiffrement` et `recv_all` qui agissent de la même façon que pour les routeurs. Il y a ensuite la partie qui permet de construire l'oignon avec la fonction `construire_oignon (message_final,ip_dest,port_dest, chemin)` qui fabrique l'oignon de l'intérieur vers l'extérieurs en chiffrant successivement avec `n` de chaque routeur, ce qui permet une bonne transmission des messages vers le clients voulu.

- Partie Interface :

La partie Interface utilise beaucoup de champs à compléter sur l'interface. Il y a les champs à remplir telles que l'IP et le port du serveur master ainsi que le port d'écoute, il y a ensuite la liste des routeurs ainsi que des clients qui s'affiche et s'actualise régulièrement, il y a ensuite le choix du mode de routage avec le mode aléatoire et sélection, le mode aléatoire choisi aléatoirement les routeurs dans la liste pour les sauts (le nombre choisi dépend du nombre de sauts voulu) et le mode sélection permet de sélectionner en cliquant sur les routeurs voulu dans la liste des routeurs sur l'interface, il y a ensuite le choix du nombre de saut qui est de base à 3 mais qui peut être changé sur l'interface avec les flèches sur l'interface, il y a aussi l'option de timeout qui permet juste que si le premier message vers le routeur d'entrée ne renvoie pas une réponse à partir d'un certain temps spécifié de réessayer en changeant l'oignon pour sauter le routeur défaillant et de renvoyer un message, il y a ensuite une zone de log qui permet d'afficher tout les retour des routeurs et les messages des clients avec lesquels nous pouvons communiquer, il y a ensuite une zone pour inscrire les informations de la destinations du message ainsi que le message. Les fonctions qui sont utilisés sont `on_connect_master()` qui démarre le listener au master et démarre le listener local, il se connecte ensuite au master en tant que client b'C' + `ip_local:port_local` et lance `thread_reception_master` qui permet de recevoir continuellement les informations actuelle du master, il y a aussi la fonction `on_send_message()` qui récupère le destinataire inscrit dans l'interface puis construit un chemin de routeurs avec le mode sélectionné (aléatoire ou via la sélection) puis construit l'oignon avec l'encryption nécessaire pour le bon fonctionnement du décryptage puis gère les tentatives de connexion au routeur d'entrée avec des timeout et retries si besoin lors de l'envoi du message par le client.

- Partie Threads Réseau :

Il y a plusieurs fonctions qui permettent la communication avec des threads, `thread_reception_master(sock_master>window)` permet de recevoir périodiquement les informations sur les routeurs et les clients et de mettre à jour les listes des routeurs et clients, il y a aussi `thread_comm_avec_routeur(sock_master>window)` qui permet de lire les réponses ou fermetures du routeur d'entrée et les logs, il y a aussi `thread_listener_client(listen_ip,listen_port>window)` qui permet de recevoir les messages du routeur de sortie et afficher [FINAL ip:port] message, puis la fonction `thread_relais_client(listen_ip,listen_port>window)` et `handle_hop_clients(...)` qui servent de petit relai local capable d'agir comme intermédiaire ou sortie pour un message chiffré avec une clé simple `n_routeur` (ils ne sont pas enregistré au master donc ils ne sont pas sensé être utilisé (juste de la redondance au cas où)).

5. Récapitulatif des fonctions

- Master :

`handle_routeur(conn, addr)` : Fonction d'enregistrement de routeur.

`handle_client(conn, addr)` : Fonction d'enregistrement de client.

`broadcast_routeurs_et_clients()` : Fonction de diffusion de la topologie aux clients.

- Routeur :

`envoyer_cle_publique_au_master(ip_master, port_master, cle_pub, listen_port)` : Fonction d'annonce au master.

handle_client(conn, addr, log_fn) : Fonction de traitement d'un message de routage.

- Client :

construire_oignon(message_final, ip_dest, port_dest, chemin) : Fonction de création du message chiffré pour le réseau.

on_send_message() : point d'entrée utilisateur pour envoyer un message.

thread_listener_client(...) : Fonction de réception de messages finaux.

6. Description des protocoles réseaux:

1. Enregistrement auprès du master

- Un routeur ou pseudo-routeur se connecte à master_ip:master_port :
 - envoie 1 octet b'R'.
 - envoie 4 octets (big endian) = longueur du payload.
 - envoie le payload texte :
 - "{listen_port};{e};{n}".
- Un client se connecte à master_ip:master_port :
 - envoie 1 octet b'C'.
 - envoie 4 octets (big endian) = longueur de l'identifiant.
 - envoie la chaîne :
 - "ip_client:port_local".

2. Diffusion des listes aux clients

Le master construit un message texte :

- ligne routeurs :

ROUTEURS:ip1;port1;e1;n1|ip2;port2;e2;n2|...

- ligne clients :

CLIENTS:pseudo1|pseudo2|...

Il concatène avec \n, encode en UTF-8, et envoie à chaque client :

- 4 octets longueur totale,
- puis le payload.

Chaque client parse ces deux blocs pour remplir ses listes.

3. Routage en oignon

- Le client choisit un chemin de routeurs [r1, r2, ..., rk] avec leurs ip, port, n.
- Construit l'oignon :

- couche interne : message_final|ip_dest:port_dest chiffrée avec n_rk.
- pour chaque routeur précédent ri :
 - nouvelle couche = ip_ri+1:port_ri+1|ancienne_couche chiffrée avec n_ri.
- Le client se connecte à r1.ip:r1.port et envoie :
 - 4 octets longueur,
 - la chaîne chiffrée finale.

Chaque routeur :

- lit 4 octets (longueur), lit le message,
- déchiffre avec simple_dechiffrement(..., n_routeur),
- sépare first_field|rest :
 - si first_field contient ip_suiv:port_suiv → hop intermédiaire :
 - ouvrir une connexion vers ce ip_suiv:port_suiv,
 - envoyer rest précédé de sa longueur,
 - renvoyer b"OK" à l'expéditeur.
 - sinon → sortie :
 - first_field = message final, rest contient ip_dest:port_dest,
 - ouvrir une connexion vers ip_dest:port_dest (le client destinataire),
 - lui envoyer message_final avec longueur.

Le client destinataire reçoit sur son listener final et affiche.

7. Description du cryptage :

Principe de l'algorithme :

- Chaque routeur possède un paramètre n (issu de la génération RSA pédagogique, mais utilisé ici seulement comme graine de chiffrement).
- Le chiffrement utilisé dans le projet est un chiffrement par décalage pseudo-aléatoire sur chaque caractère, défini par :

python

```
def simple_chiffrement(message, n):
```

```
    random.seed(n)
```

```

encrypted = ""
for c in message:
    shift = random.randint(1, 10)
    encrypted += chr((ord(c) + shift) % 256)
return encrypted

```

- Le déchiffrement refait les mêmes tirages pseudo-aléatoires, mais en soustrayant le décalage :

python

```

def simple_dechiffrement(encrypted, n):
    random.seed(n)
    decrypted = ""
    for c in encrypted:
        shift = random.randint(1, 10)
        decrypted += chr((ord(c) - shift) % 256)
    return decrypted

```

- Comme `random.seed(n)` est appelé au début de chaque fonction, la séquence de décalages est déterministe pour un même `n` et un même ordre de caractères, ce qui permet de retrouver le message d'origine.

Dans le routage en oignon :

- Le client construit d'abord la couche interne `message_final|ip_dest:port_dest` et la chiffre avec le `n` du dernier routeur de la route.
- Puis il ajoute successivement des couches `ip_suiv:port_suiv|...` et les chiffre avec le `n` de chaque routeur intermédiaire, de l'intérieur vers l'extérieur.
- Chaque routeur :
 - déchiffre la couche la plus externe avec son propre `n`,
 - lit soit l'adresse du prochain hop, soit le destinataire final.

Forces de cet algorithme :

Simplicité de mise en œuvre : l'algorithme est très court, facile à comprendre, et ne dépend pas de bibliothèques cryptographiques externes.

- Adapté à la pédagogie :
 - illustre clairement le principe "clé partagée par routeur" (via le paramètre `n`),

- permet de comprendre le routage en oignon (superposition de couches chiffrées) sans complexité mathématique lourde.
- Intégration aisée au protocole :
 - fonctionne sur de simples chaînes de caractères ("`champ1 | champ2`"),
 - n'impose pas de format binaire compliqué.
- Performance suffisante pour un TP : le chiffrement/déchiffrement est très rapide même en Python pur.

Faiblesses et limites de sécurité :

- Non cryptographiquement sûr :
 - l'algorithme est un simple décalage pseudo-aléatoire par caractère, pas un chiffrement standard comme AES ou ChaCha20.
 - il n'y a aucune analyse de sécurité formelle ; un attaquant motivé pourrait casser la clé en observant quelques messages.
- Graine fixe et déterministe :
 - l'utilisation de `random.seed(n)` implique que pour un même `n`, la séquence de décalages est toujours la même.
 - Il n'y a pas d'IV (vecteur d'initialisation) ni de sel, donc deux messages identiques chiffrés avec le même `n` produiront des schémas de décalage identiques.
- Absence d'authentification :
 - L'algorithme ne fournit aucune preuve d'intégrité ni d'authenticité des messages.
 - Un attaquant sur le réseau pourrait modifier des octets du message chiffré sans être détecté par les routeurs ou les clients.
- Taille de clé limitée :
 - même si `n` provient d'un RSA pédagogique, il est généré sur de petites tailles (16 bits dans ton code), largement insuffisantes pour un environnement réel.
- Pas de protection contre l'analyse de trafic :
 - le chiffrement masque le contenu, mais pas la longueur ni le timing des messages ; combiné à la simplicité de l'algorithme, cela ne résiste pas à un attaquant fort.

Conclusion sur le chiffrement :

L'algorithme est adapté au chiffrement demandé pour montrer le principe de routage en oignon et de la séparation des rôles mais il ne peut pas être utilisé en production ou dans un vrai système car il manque de véritables mécanismes de protection et le chiffrement est trop faible pour une utilisation réelle.

8. Gestion du projet :

1	Planning :	17/11	25/11	26/11	28/11	05/12	09/12	10/12	11/12	12/12	15/12	16/12	17/12	18/12	19/12
2	Compréhension de l'architecture demandé														
3	Mise en place du serveur														
4	Mise en place des clients														
5	Mise en place des routeurs														
6	Mise en place du cryptage														
7	Création de la base de données														
8	Mise en place des machines virtuelles pour les tests														
9	Faire les livrables														

Le planning a été respecté et le projet a été fini à temps.

9. Analyse des compétences :

AC23.01 – Automatiser l'administration système avec des scripts

Dans ce projet, l'administration du réseau et de la base n'est pas faite à la main routeur par routeur :

- Script serveur_Master_Qt.py qui centralise l'enregistrement automatique des routeurs et des clients, met à jour la BDD et nettoie périodiquement les entrées inactives (thread cleanup_inactifs).
- Utilisation de commandes automatisées pour sauvegarder / restaurer la base (mysqldump, import du dump) intégrées dans la procédure d'installation.

AC23.02 – Développer une application à partir d'un cahier des charges

Le projet répond à un cahier des charges clair :

- concevoir une architecture distribuée avec routage en oignon,
- fournir une interface utilisateur Qt pour le master, les routeurs et les clients,
- permettre la sélection du chemin, la gestion des timeouts, l'intégration BDD, etc.

J'ai su :

- traduire les besoins (anonymisation, routage multi-sauts, visibilité des routeurs/clients) en fonctionnalités,
- organiser le code en modules (master, routeur, client) avec GUI, logique réseau et accès BDD séparés.

AC23.03 – Utiliser un protocole réseau pour programmer une application client/serveur

Le cœur du projet, est une application client/serveur basée sur TCP :

- communication master ↔ routeurs ↔ clients via sockets TCP,
- protocole applicatif par : octet d'identification ('R' ou 'C'), longueur sur 4 octets, puis messages texte (ROUTEURS:..., CLIENTS:..., oignon chiffré, etc.).
- gestion multi-clients et multi-routeurs avec threads pour accepter plusieurs connexions.

AC23.04 – Installer, administrer un système de gestion de données

J'ai :

- installé MariaDB sous Windows / Linux / macOS,
- créé la base r309_onion avec ses tables routeurs et clients,

- gérer les utilisateurs BDD (création de r309_user, droits sur la base),
- mis en place la sauvegarde / restauration via mysqldump,
- intégré la mise à jour des colonnes date_* et dernier_heartbeat pour suivre l'activité.

AC23.05 – Accéder à un ensemble de données depuis une application

Le master accède à la BDD MariaDB pour :

- insérer / mettre à jour les routeurs et les clients,
- supprimer les entrées inactives,
- éventuellement réutiliser ces données pour l'affichage et la supervision.

CE3.01 – Être à l'écoute des besoins du client

J'ai suivi le cahier des charges demandé car j'ai :

- pris en compte les besoins de routage en oignon, de visualisation des routeurs et clients, de chiffrement même simplifié,
- adapté certaines fonctionnalités (ex. ajout du timeout configurables, mode de sélection des routeurs) pour rendre l'outil plus utilisable.

CE3.02 – Documenter le travail réalisé

Mon rapport contient :

- description de l'architecture,
- structure du code (master, routeur, client),
- description des protocoles,
- description de l'algorithme de chiffrement (forces/faiblesses),
- guides d'installation (Windows, Linux, macOS) et d'utilisation.

CE3.03 – Utiliser les outils numériques à bon escient

J'utilises plusieurs outils adaptés au contexte :

- GitHub pour versionner et partager le code,
- PyQt5 pour construire rapidement des interfaces graphiques,
- MariaDB pour stocker la topologie du réseau,
- environnement virtuel Python pour isoler les dépendances,
- VM Linux pour tester sur plusieurs OS.

CE3.04 – Choisir les outils de développement adaptés

J'ai utilisé différents outils :

- Python : rapide pour prototyper réseau + BDD + GUI,
- sockets TCP : contrôle fin du protocole, bon pour un projet pédagogique de réseaux,
- Qt : permet d'avoir un retour visuel (logs, listes de routeurs/clients),
- MariaDB : SGBD classique facilement installable sur plusieurs OS.

CE3.05 – Intégrer les problématiques de sécurité

Même si le chiffrement est simple, j'ai :

- mis en place un routage en oignon pour limiter la visibilité du chemin (chaque routeur ne connaît que le précédent et le suivant),
- séparer les rôles (master, routeurs, clients) pour éviter qu'un seul composant voit tout,
- analysé les faiblesses de ton algorithme (clé faible, pas d'authentification, pas d'IV) et expliqué qu'il ne convient pas à la production.

10. Conclusion :

Ce projet m'a permis de mieux comprendre les communications TCP ainsi que le principe du cryptage. J'ai choisi de travailler seul car je pensais que je pouvais faire ce travail tout seul, ce que j'ai réussi car le projet est fini et rendu à temps. Cette SAE m'a permis de concevoir et de réaliser une architecture complète de communication anonyme (master, routeurs, clients, BDD). j'ai mis en œuvre un routage en oignon, un chiffrement dédié, une base MariaDB multi-OS et des interfaces Qt utilisables. Les points moins positifs est que l'idée de faire un client-routeur n'a pas été possible et que le chiffrement n'est pas très utile car le cryptage n'est pas très avancé et n'est pas suffisant pour une vraie utilisation.

Le projet reste très éducatif et m'a permis de beaucoup apprendre.